Introduction:

The application allows users to register, log in, view all registered users, select another user, exchange one-to-one messages, and reload previous message history from the database.

The project was developed on an Arch Linux system. Since MariaDB is installed locally on this PC and is compatible with JDBC, the chat server connects to MariaDB through the MariaDB Java client driver.

Project Structure:

The project is a Maven Java project with one server program and one JavaFX client program.

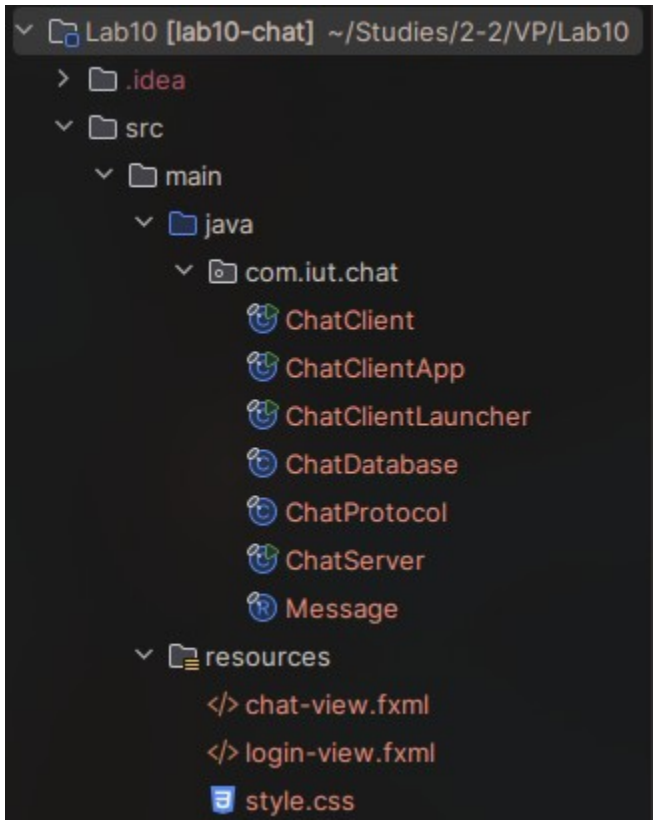

Fig: screenshot of file structure

ChatServer.java starts the socket server on port 5001. It accepts several clients at the same time, creates a separate ClientSession for every connected client, and uses a thread pool to process clients concurrently.

ChatClient.java is a normal Java launcher class. It starts ChatClientApp.java, which is the JavaFX Application class. This separate launcher avoids the JavaFX runtime error that can happen when IntelliJ runs a JavaFX Application class directly without module path settings.

ChatClientApp.java loads the FXML files from the resources folder, connects UI controls to event handlers, sends protocol commands to the server, and updates the chat window when messages or history lines are received.

ChatDatabase.java manages all MariaDB operations. It creates the tables, registers users, authenticates passwords, returns the user list, saves messages, and loads chat history.

ChatProtocol.java contains the text protocol helper methods. Message.java is a record class used for transferring message data inside the server code.

FXML Files:

The resources folder contains two FXML files that can be opened and edited in Scene Builder:

1. login-view.fxml
2. chat-view.fxml

login-view.fxml contains the login/register screen. chat-view.fxml contains the main chat screen with the registered user list, message history area, message input field, and send button. The Java code accesses controls through fx:id values, so the fx:id values should not be removed while editing the layout in Scene Builder.

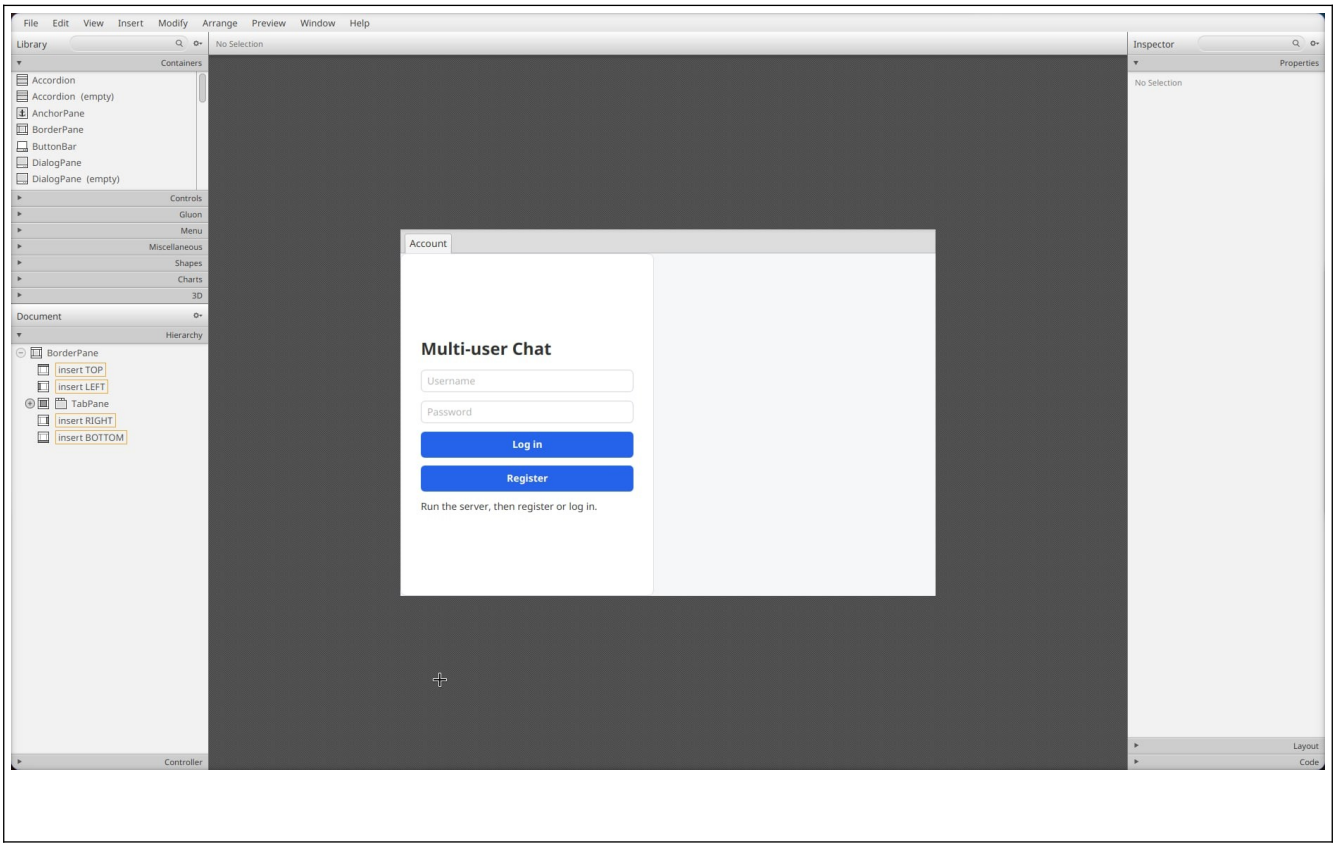

Fig: login-view.fxml opened in Scene Builder

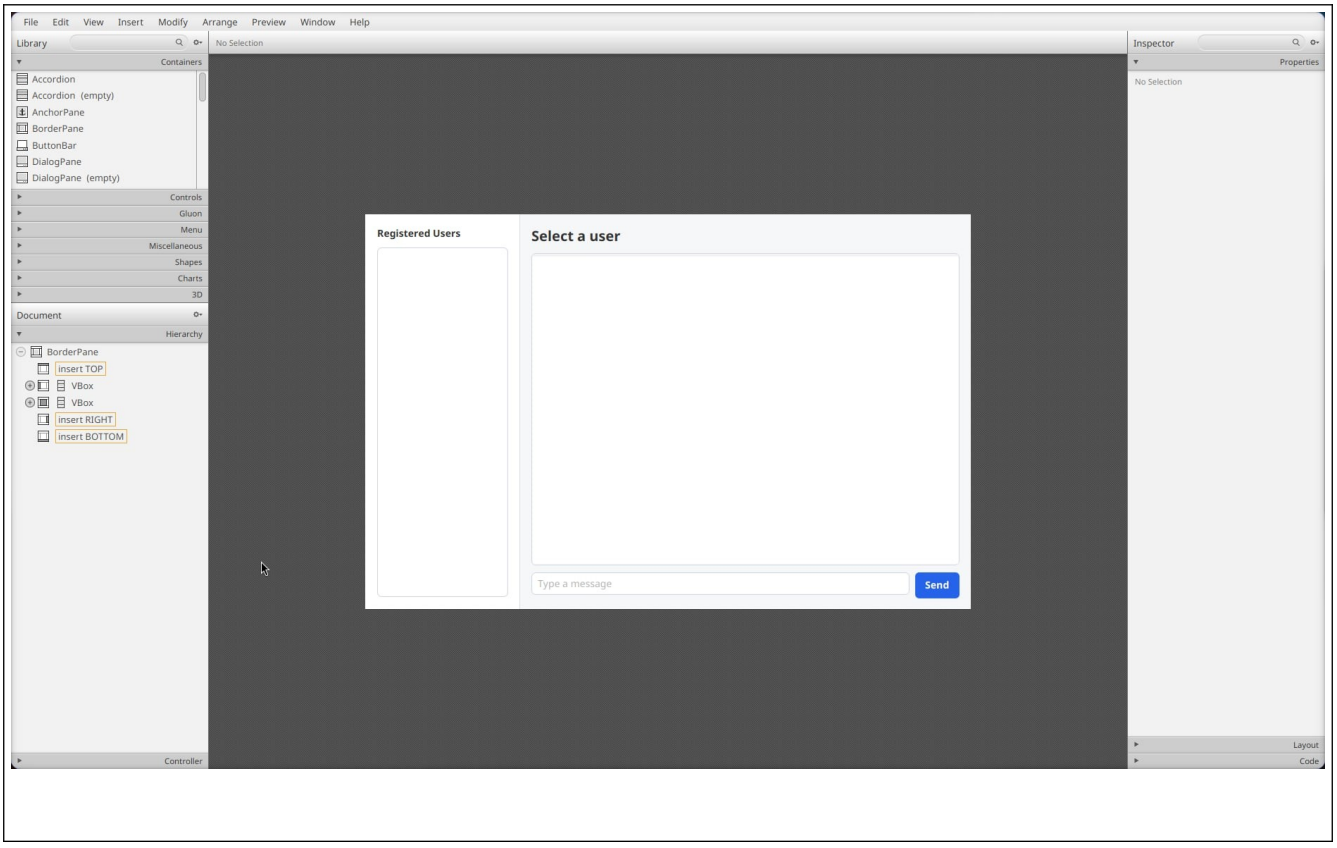

Fig: chat-view.fxml opened in Scene Builder

Application Screenshots:

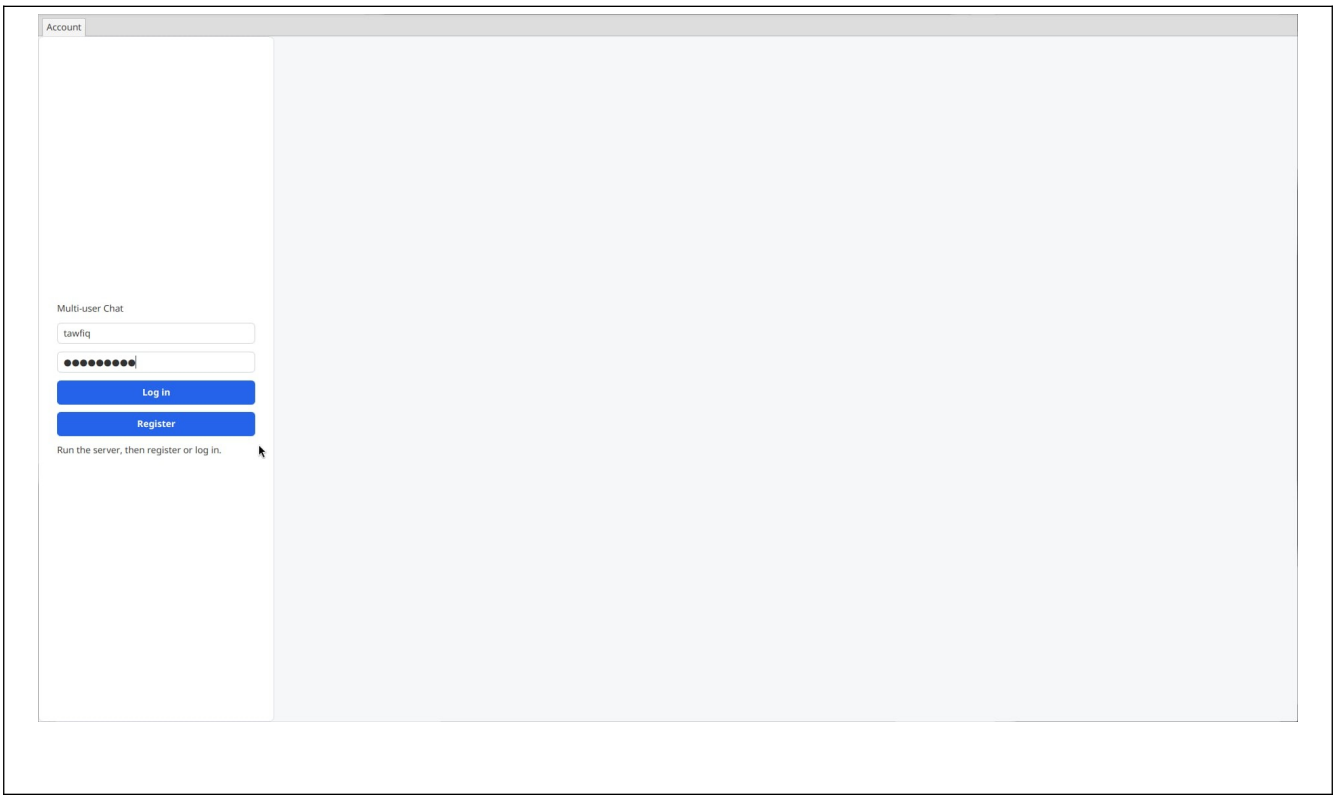

Fig: login page

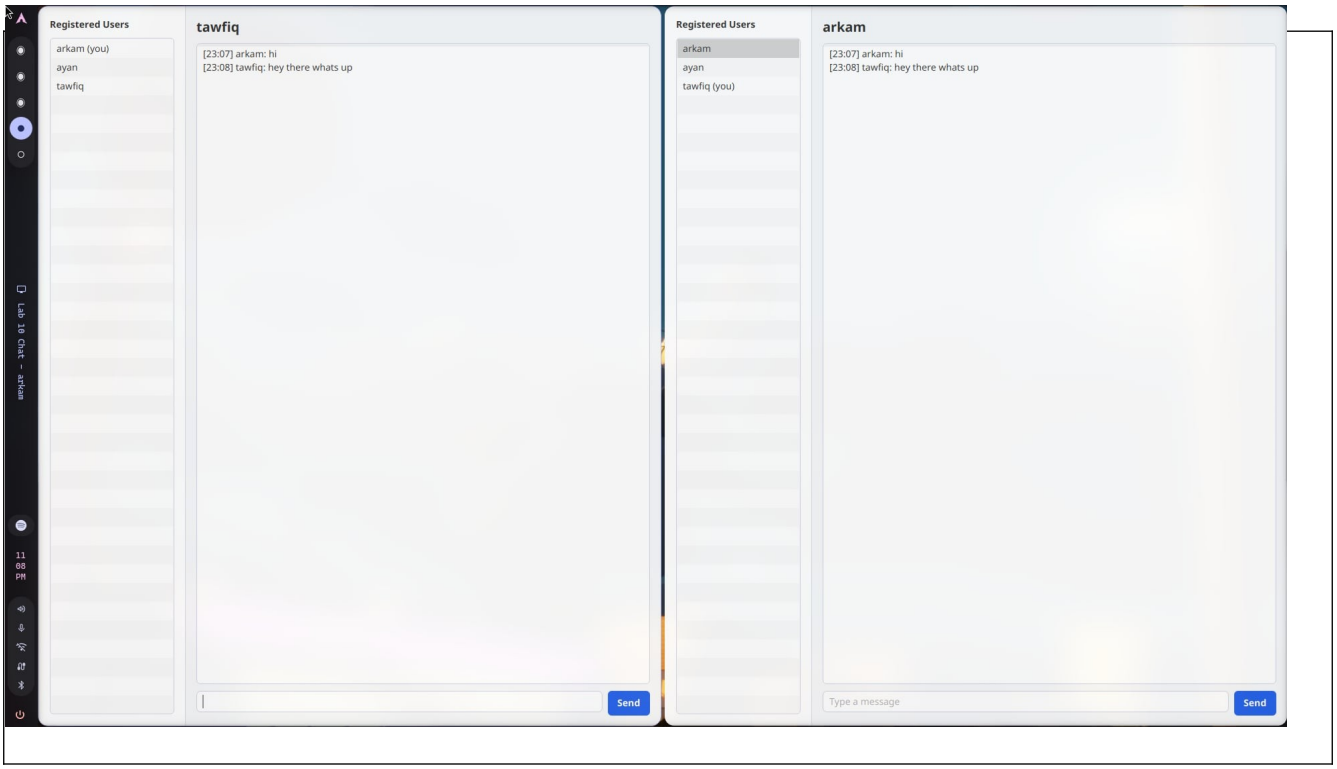

Fig: chat screen between two users

MariaDB Schema:

The application uses the MariaDB server installed on this PC. The database connection configuration is as follows:

URL: jdbc:mariadb://localhost:3306/test

User: tawfiq

Password: empty

The server automatically creates the required tables when ChatServer starts.

1 . lab10_chat_users table

Column	Data Type	Description
id	BIGINT AUTO_INCREMENT PRIMARY KEY	Internal user row id.
username	VARCHAR(20) UNIQUE NOT NULL	Login name of the user.
password_hash	CHAR(64) NOT NULL	SHA-256 hash of the salted password.
salt	CHAR(32) NOT NULL	Random salt used during password hashing.
created_at	BIGINT NOT NULL	Account creation time in milliseconds.

Fig: Columns of lab10_chat_users table

2 . lab10_chat_messages table

Column	Data Type	Description
id	BIGINT AUTO_INCREMENT PRIMARY KEY	Internal message row id.
sender	VARCHAR(20) NOT NULL	Username of the sender.
recipient	VARCHAR(20) NOT NULL	Username of the recipient.
body	TEXT NOT NULL	Message text.
sent_at	BIGINT NOT NULL	Message time in milliseconds.

Fig: Columns of lab10_chat_messages table

Data Entry in Database Table

username	created_at
ayan	1785083463512
tawfiq	1785083420835

sender	recipient	body	sent_at
ayan	tawfiq	hi	1785083484293
tawfiq	ayan	hello there	1785083501530

Fig: Data entries in the MariaDB chat tables

Code Snippets:

ChatDatabase.java

```
private static final String DEFAULT_DB_URL = "jdbc:mariadb://localhost:3306/test";
private static final String DEFAULT_DB_USER = "tawfiq";
private static final String DEFAULT_DB_PASSWORD = "";
private static final String DB_URL = setting("CHAT_DB_URL", DEFAULT_DB_URL);
private static final String DB_USER = setting("CHAT_DB_USER", DEFAULT_DB_USER);
private static final String DB_PASSWORD = setting("CHAT_DB_PASSWORD",
DEFAULT_DB_PASSWORD);

private Connection connect() throws SQLException {
    return DriverManager.getConnection(DB_URL, DB_USER, DB_PASSWORD);
}
```

This part contains the database configuration. The server connects to the locally installed MariaDB server through JDBC. The environment variables allow the database URL, username, and password to be changed without editing the source code.

```
statement.executeUpdate("""
        CREATE TABLE IF NOT EXISTS lab10_chat_users (
            id BIGINT PRIMARY KEY AUTO_INCREMENT,
            username VARCHAR(20) NOT NULL UNIQUE,
            password_hash CHAR(64) NOT NULL,
            salt CHAR(32) NOT NULL,
            created_at BIGINT NOT NULL
        ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
        """);
statement.executeUpdate("""
        CREATE TABLE IF NOT EXISTS lab10_chat_messages (
            id BIGINT PRIMARY KEY AUTO_INCREMENT,
            sender VARCHAR(20) NOT NULL,
            recipient VARCHAR(20) NOT NULL,
            body TEXT NOT NULL,
            sent_at BIGINT NOT NULL,
            FOREIGN KEY (sender) REFERENCES lab10_chat_users(username),
            FOREIGN KEY (recipient) REFERENCES lab10_chat_users(username),
            INDEX idx_lab10_messages_pair (sender, recipient, sent_at)
        ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
        """);
```

This creates the users and messages tables if they are not already present. The messages table stores sender and recipient usernames and uses foreign keys to connect them to registered users.

```
synchronized Message saveMessage(String sender, String recipient, String body) {
    long sentAt = System.currentTimeMillis();
    try (Connection connection = connect();
         PreparedStatement statement = connection.prepareStatement(
                 "INSERT INTO lab10_chat_messages(sender, recipient, body,
sent_at) VALUES (?, ?, ?, ?)")) {
        statement.setString(1, sender);
        statement.setString(2, recipient);
        statement.setString(3, body);
        statement.setLong(4, sentAt);
        statement.executeUpdate();
        return new Message(sender, recipient, body, sentAt);
    } catch (SQLException ex) {
        throw new IllegalStateException("Could not save message", ex);
    }
}
```

This method saves every sent message in MariaDB. PreparedStatement is used so that user input is not directly concatenated into SQL.

ChatServer.java

```
private final Map<String, ClientSession> onlineClients = new
ConcurrentHashMap<>();
private final ExecutorService clientPool = Executors.newCachedThreadPool();

public static void main(String[] args) {
    new ChatServer().start();
}

private void start() {
    System.out.println("Chat server listening on port " + ChatProtocol.PORT);
    try (ServerSocket serverSocket = new ServerSocket(ChatProtocol.PORT)) {
        while (true) {
            Socket socket = serverSocket.accept();
            clientPool.submit(new ClientSession(socket));
        }
    } catch (IOException ex) {
        System.err.println("Server stopped: " + ex.getMessage());
    }
}
```

This is how the server handles multiple clients. The server accepts a socket connection and immediately submits a new ClientSession to a thread pool. Therefore one connected client does not block the server from accepting other clients.

```
if (!database.authenticate(requestedUsername, password)) {
    send(ChatProtocol.command("AUTH_FAIL", ChatProtocol.encode("Wrong username or
password.")));
    return;
}
username = requestedUsername;
ClientSession previous = onlineClients.put(username, this);
if (previous != null && previous != this) {
    previous.send(ChatProtocol.command("ERROR", ChatProtocol.encode("You signed in
from another window.")));
    previous.close();
}
send(ChatProtocol.command("AUTH_OK", ChatProtocol.encode(username)));
broadcastUsers();
```

This is the login handshake. The client sends a LOGIN command containing the encoded username and password. The server checks the password using ChatDatabase. If the login is valid, the server stores the username inside that ClientSession and adds the session to the onlineClients map. From this point, that socket connection is identified by its username.

```
Message message = database.saveMessage(username, recipient, body);
String messageLine = messageLine(message);
send(messageLine);
ClientSession recipientClient = onlineClients.get(recipient);
if (recipientClient != null) {
    recipientClient.send(messageLine);
}
```

This is the message routing logic. The server saves the message first. Then it sends the saved message back to the sender and also sends it to the recipient if the recipient is currently online. If the recipient is offline, the message still remains stored in MariaDB and will be loaded later from history.

```
send(ChatProtocol.command("HISTORY_BEGIN", ChatProtocol.encode(otherUser)));
List<Message> history = database.history(username, otherUser);
for (Message message : history) {
    send(messageLine(message));
}
send(ChatProtocol.command("HISTORY_END", ChatProtocol.encode(otherUser)));
```

This loads previous conversation history. When the client selects another user, it sends a HISTORY command. The server queries all messages where the two users are sender and recipient in either order, sorts them by sent_at, and sends them back to the client.

ChatClientApp.java

```
private Parent loginView() {
    Parent root = loadView("/login-view.fxml");
    usernameField = lookup(root, "usernameField");
    passwordField = lookup(root, "passwordField");
    statusLabel = lookup(root, "statusLabel");

    Button loginButton = lookup(root, "loginButton");
    loginButton.setDefaultButton(true);
    loginButton.setOnAction(event -> sendAuth("LOGIN"));

    Button registerButton = lookup(root, "registerButton");
    registerButton.setOnAction(event -> sendAuth("REGISTER"));
    return root;
}
```

The client UI is loaded from FXML. The Java code looks up controls by fx:id and attaches event handlers to the login and register buttons.

```
private void selectConversation(String user) {
    if (user == null || user.equals(currentUser)) {
        return;
    }
    selectedUser = user;
    conversationTitle.setText(user);
    messageField.setDisable(false);
    chatArea.setText(conversations.getOrDefault(user, new
StringBuilder()).toString());
    out.println(ChatProtocol.command("HISTORY", ChatProtocol.encode(user)));
}
```

When a user is selected from the left panel, the client records that user as the current recipient and requests conversation history from the server.

Handling Multiple Clients

The server uses ServerSocket to listen on port 5001. Every time a client connects, the server creates a ClientSession object for that socket. The ClientSession is submitted to an ExecutorService created with Executors.newCachedThreadPool(). Because every session runs on a separate worker thread, multiple clients can stay connected and send commands at the same time.

Identification of Users during Login

Before logging in, the server only knows that a socket is connected. The client sends REGISTER or LOGIN commands with encoded username and password fields. During login, ChatDatabase authenticates the username and password. If authentication succeeds, the server stores that username in the ClientSession and puts the session in onlineClients using the username as the key. This creates the connection between a socket and a user identity.

Routing Messages

The client sends a SEND command with the selected recipient and the message body. The server checks that the sender is logged in, the recipient exists, the recipient is not the sender, and the message is not blank. It then saves the message to MariaDB. After saving, it sends the message back to the sender and also looks up the recipient in onlineClients. If the recipient is online, the server sends the same message to the recipient's session. If the recipient is offline, no message is pushed immediately, but the database still stores it.

Message History

Every message is inserted into lab10_chat_messages with sender, recipient, body, and sent_at. When a client selects a user, it sends a HISTORY command to the server. The server loads all rows where the two users appear as sender and recipient in either direction. The query orders messages by sent_at, so the conversation appears chronologically. The server wraps the result between HISTORY_BEGIN and HISTORY_END, and the client uses those protocol lines to refresh the chat area.